Assignment 1 Report

Name: Shirou Jing
NetID: sj186

Oct. 20, 2026

1 Experiment

Goal. Part 1 implements a fully connected network in NumPy with hand-written backpropagation, first as the 3-layer network of the starter code and then as an n-layer network, and trains it on Make-Moons and on the scikit-learn Digits dataset. Part 2 trains the DCN conv(5-5-1-32)–ReLU–maxpool–conv(5-5-32-64)–ReLU–maxpool–fc(1024)–ReLU–dropout(0.5)–softmax(10) on MNIST in PyTorch, monitors weights, biases, net inputs and activations in TensorBoard, and compares activation functions, initializations and optimizers.

2 Experiment Setup

All experiments run on NERSC Perlmutter login nodes. Part 1 is NumPy on CPU; Part 2 runs on one A100 per run.

Table 1: Environment.

Item	Value
System	NERSC Perlmutter, login node
GPU	NVIDIA A100-PCIE-40GB; driver 580.178.04
CPU	AMD EPYC 7713 64-Core Processor
OS	SUSE Linux Enterprise Server 15 SP6
Software	NERSC module `pytorch/2.13.0`: Python 3.12.13, PyTorch 2.13.0 (CUDA 13.0), torchvision 0.28.0, NumPy 2.4.4, scikit-learn 1.9.0, TensorBoard 2.21.0

3 Results and Discussion

3.1 Make-Moons Dataset

`generate_data()` draws 200 points from `make_moons` with noise 0.2 (Figure 1). The two classes interleave, so no linear boundary separates them.

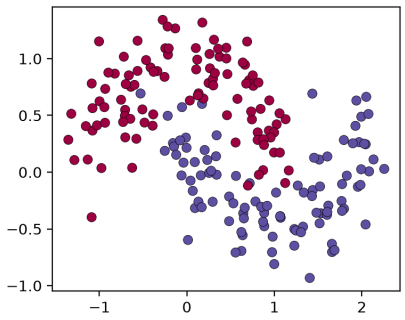


Figure 1: Make-Moons, 200 samples, noise 0.2.

3.2 Activation Functions and Their Derivatives

With $\sigma(z) = 1/(1+e^{-z})$:

$$\frac{d}{dz}\tanh z = \frac{(e^z+e^{-z})^2-(e^z-e^{-z})^2}{(e^z+e^{-z})^2} = 1-\tanh^2 z,$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\big(1-\sigma(z)\big),$$

$$\frac{d}{dz}\max(0,z) = \begin{cases} 1 & z>0 \\ 0 & z<0 \end{cases}$$

The ReLU derivative is undefined at $z=0$; the code uses 0 there. `actFun` and `diff_actFun` implement these expressions directly.

3.3 Forward Pass and Loss

Samples are stacked as rows of $X \in \mathbb{R}^{N\times 2}$, so the code computes $z_1 = XW_1 + b_1$, $a_1 = f(z_1)$, $z_2 = a_1W_2 + b_2$ and $\hat{Y} = \mathrm{softmax}(z_2)$ row-wise, which is Eqs. (1)–(4) transposed. The softmax subtracts the row maximum before exponentiating. `calculate_loss` returns Eq. (5) plus the starter code's L2 term:

$$L = -\frac{1}{N}\sum_n \log \hat{y}_{n,y_n} + \frac{\lambda}{2N}\big(\|W_1\|^2 + \|W_2\|^2\big).$$

3.4 Backpropagation

For one sample, $\partial \hat{y}_j/\partial z_{2,k} = \hat{y}_j(\delta_{jk} - \hat{y}_k)$, so

$$\frac{\partial}{\partial z_{2,k}}\Big(-\sum_j y_j \log \hat{y}_j\Big) = -\sum_j y_j(\delta_{jk}-\hat{y}_k) = \hat{y}_k - y_k,$$

using $\sum_j y_j = 1$. Over the batch, $\delta_2 \equiv \partial L/\partial z_2 = \frac{1}{N}(\hat{Y}-Y)$. The chain rule then gives

$$\frac{\partial L}{\partial W_2} = a_1^\top \delta_2 + \frac{\lambda}{N}W_2, \qquad \frac{\partial L}{\partial b_2} = \mathbf{1}^\top \delta_2,$$

$$\delta_1 \equiv \frac{\partial L}{\partial z_1} = \big(\delta_2 W_2^\top\big) \odot f'(z_1),$$

$$\frac{\partial L}{\partial W_1} = X^\top \delta_1 + \frac{\lambda}{N}W_1, \qquad \frac{\partial L}{\partial b_1} = \mathbf{1}^\top \delta_1.$$

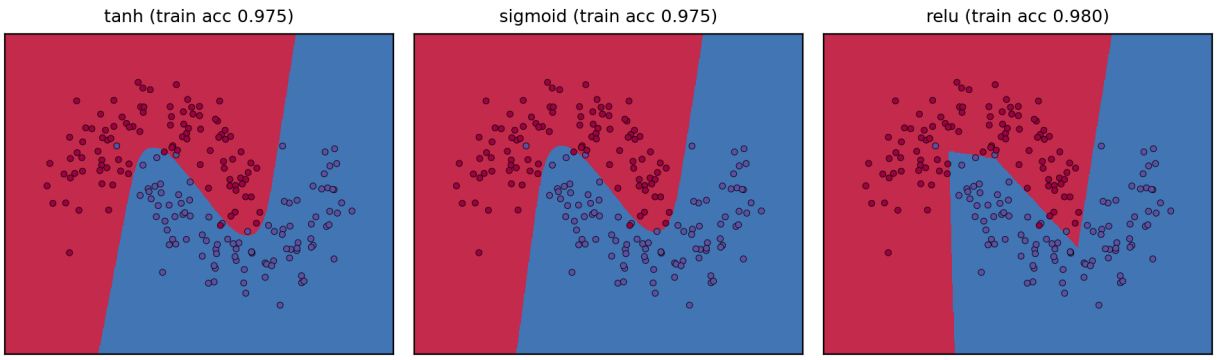


Figure 2: Decision boundaries, 3 hidden units. Test accuracy: tanh 0.967, sigmoid 0.967, ReLU 0.963.

`backprop` returns these gradients multiplied by N, i.e. summed over samples, and `fit_model` adds λW. This matches the starter's update rule, and the learning rate $\epsilon = 0.01$ absorbs the factor N. Against central finite differences, the maximum absolute gradient error is below 10^{-8} for all three activations.

3.5 Training with Tanh, Sigmoid and ReLU

Each network has 3 hidden units and is trained with the starter's settings: full-batch gradient descent, $\epsilon = 0.01$, $\lambda = 0.01$, 20,000 passes (Figure 2). Test accuracy is measured on 2,000 fresh Make-Moons samples (noise 0.2, seed 1) that are never used for training.

All three reach 97.5–98% training accuracy, and the shapes of the boundaries differ. Tanh and sigmoid give smooth curves. Sigmoid is a shifted and rescaled tanh, $\sigma(z) = \frac{1}{2}(1 + \tanh \frac{z}{2})$, so with enough training the two networks represent the same function family; sigmoid ends at a slightly higher loss (0.078 vs. 0.071). Its derivative is at most 0.25, against 1 for tanh, so with the same step size it moves more slowly. The ReLU boundary is piecewise linear with visible kinks: each hidden unit splits the plane along a line, and three units allow only a few linear pieces.

3.6 Number of Hidden Units

Tanh networks with 1–50 hidden units, same settings (Figure 3).

With 1 or 2 units the boundary is close to a line (train 0.865/0.825). Three units are enough to bend around both moons, and test accuracy is highest at 3–4 units (0.967). Beyond that, training accuracy saturates at 0.99 and the extra capacity goes into fitting individual noisy points. At 5 units an isolated island appears around a single sample; at 10–50 units the boundary curls around the outliers in the middle. Test accuracy falls to 0.946–0.955. The $\lambda = 0.01$ penalty is weak relative to the summed data loss, so it does not prevent this.

3.7 Deep Network Implementation

`n_layer_neural_network.py` follows the hints. `Layer` stores W, b and its forward cache (x, z) and implements `feedforward(x, actFun)` and `backprop(da, diff_actFun)`. The backward method returns $\partial L/\partial x$ for the layer below and stores dW and db. `DeepNeuralNetwork` subclasses `NeuralNetwork` and reuses its `actFun`, `diff_actFun`, `predict` and plotting. The constructor takes `n_layers` (counting input and output, so 3 is the network above) and `layer_size`, which is either an integer for equal widths or a list of per-layer widths. Two choices differ from the 3-layer code:

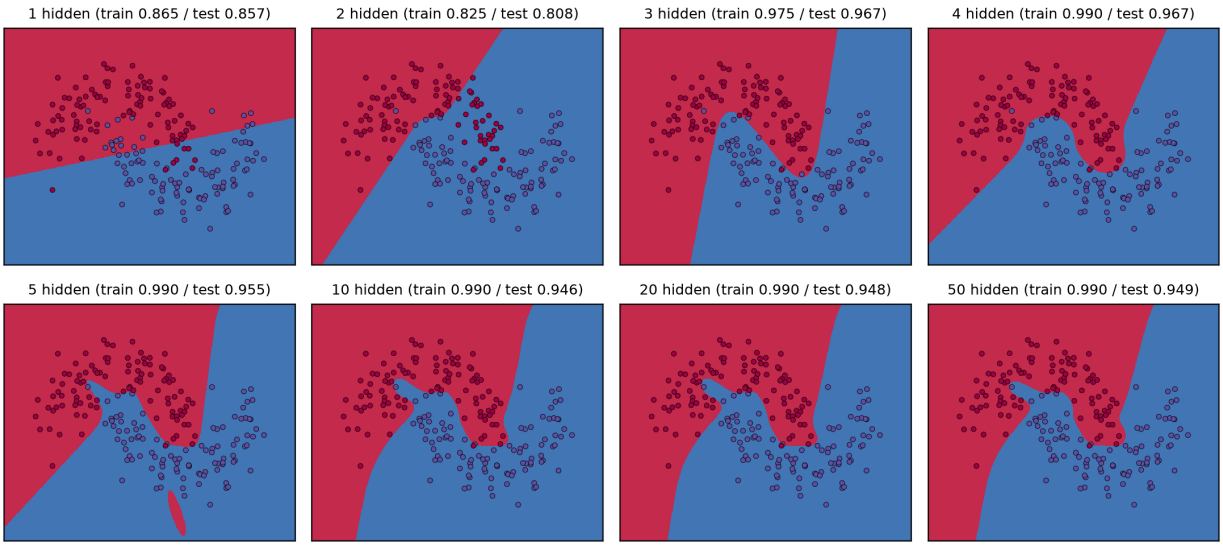


Figure 3: Tanh, varying hidden width.

- The loss averages over samples, $L = \frac{1}{N}\sum_n \mathrm{CE}_n + \frac{\lambda}{2}\sum_\ell \|W_\ell\|^2$, and the gradients are true gradients of L. The step size then does not depend on N, so the same ϵ works for 200 Moons points and 1,347 Digits samples. I use $\epsilon = 0.1$ and $\lambda = 10^{-3}$; with $\epsilon = 1.0$ the 10-layer ReLU network fails to train.
- The output layer is a `Layer` with identity activation, and softmax is applied in the network, so every layer shares one backward rule. Initialization stays $\mathcal{N}(0, 1/\text{fan-in})$ as in the starter.

The finite-difference check from Section 3.4 also passes for a 5-layer network (maximum error $\sim 10^{-10}$).

3.8 Deep Networks on Make-Moons

Figure 4 varies depth (3, 4, 6, 10 layers, width 10) and activation; Figure 5 varies width at depth 6 with tanh. All runs use 20,000 passes.

Shallow networks draw one smooth S-curve. Deeper tanh and ReLU networks produce sharper, more folded boundaries, with narrow protrusions that reach into the other class to capture single points. Training loss falls from 0.116 (3 layers) to 0.045 (10 layers) for tanh, while test accuracy drops from 0.967 to 0.954, so the extra depth mostly fits noise. ReLU boundaries stay polygonal at every depth, and the number of linear pieces grows with depth. The sigmoid network with 10 layers does not train: it predicts one class everywhere and stays at loss 0.694, about $\ln 2$. Each sigmoid layer multiplies the backward signal by $\sigma' \le 0.25$, so across eight hidden layers the derivative factors alone shrink it by up to $0.25^8 \approx 1.5\times10^{-5}$, and the first layers barely move. Tanh and ReLU at the same depth train normally. Width has a smaller effect than depth here. Width 2 already reaches 0.975 training accuracy at 6 layers, widths 10 and 50 give nearly the same boundary, and test accuracy stays at 0.956–0.959 for all widths.

3.9 Deep Networks on Digits

The second dataset is scikit-learn Digits: 1,797 grayscale 8×8 handwritten digits, 10 classes. The network has 64 inputs and 10 outputs. I use a stratified 75/25 split (1,347 / 450), standardize the features with training-set statistics, and run 5,000 full-batch passes with $\epsilon = 0.1$ and $\lambda = 10^{-3}$ (Table 2, Figure 6).

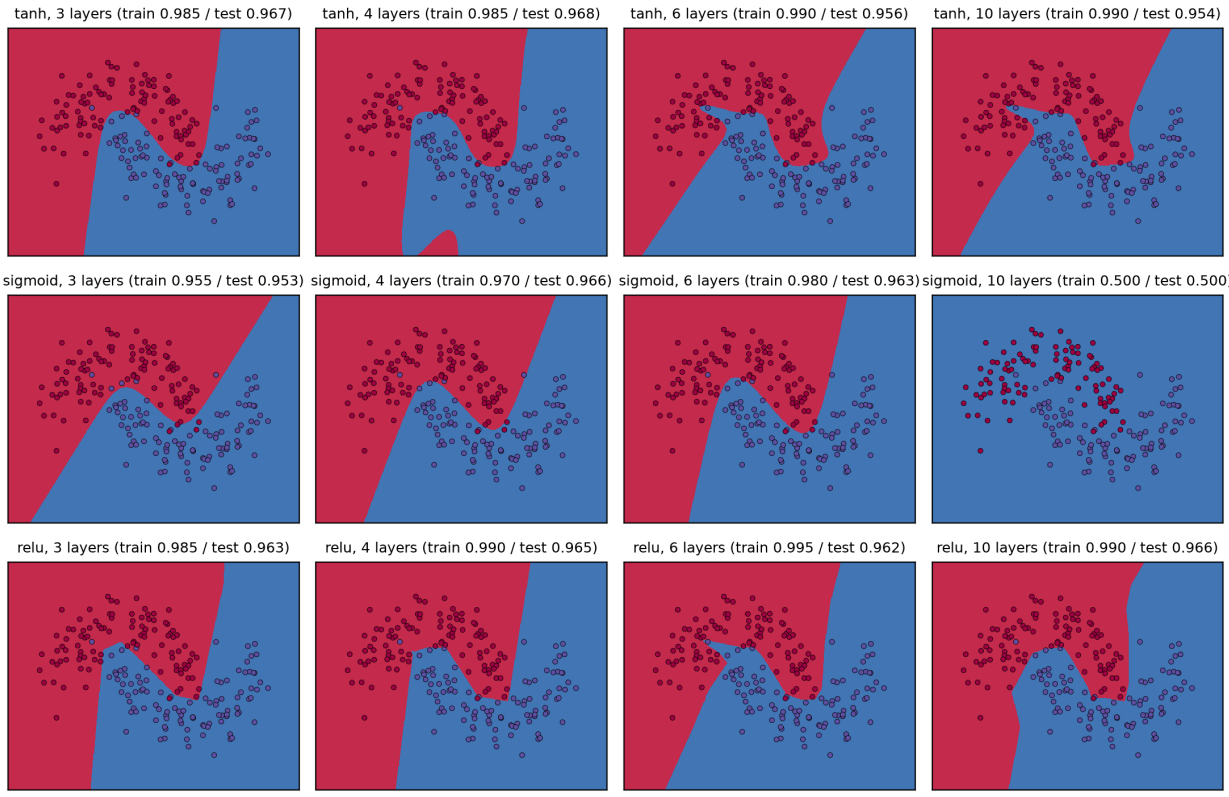


Figure 4: Depth × activation, hidden width 10. Titles give train / test accuracy.

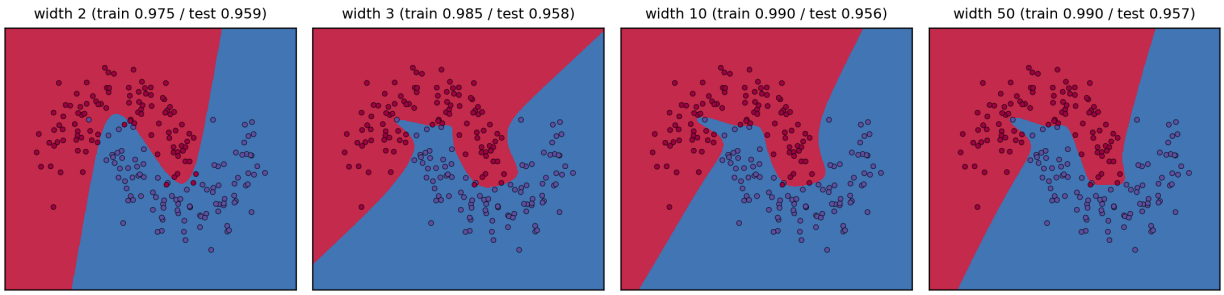


Figure 5: Width at 6 layers, tanh.

Table 2: Digits test accuracy. Every tanh and ReLU network reaches 1.000 training accuracy.

	3 layers		4 layers		6 layers	
Activation	32	128	32	128	32	128
tanh	0.980	0.978	0.978	0.969	0.980	0.976
sigmoid	0.976	0.978	0.969	0.969	0.691	0.891
ReLU	0.973	0.978	0.978	0.978	0.964	0.982

A single hidden layer already gives 97–98% test accuracy, and depth does not help: the classes are close

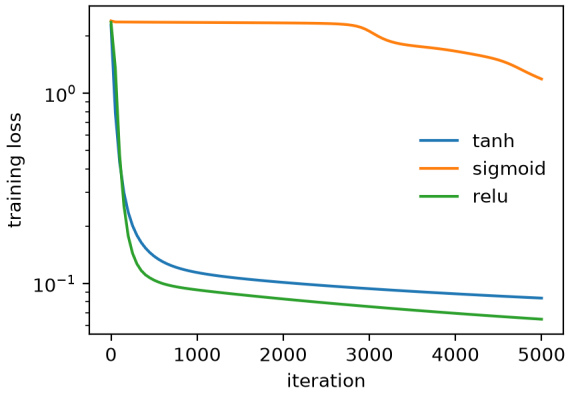


Figure 6: Digits training loss, 6 layers, width 32.

to linearly separable in 64 dimensions, and with 1,347 samples every tanh and ReLU network fits the training set perfectly. With sigmoid at 6 layers and width 32, the loss stays on a plateau near ln 10 for about 2,800 iterations before it starts to fall (Figure 6), and after 5,000 passes it reaches only 70.6% training and 69.1% test accuracy. The plateau has the same cause as the 10-layer Moons failure: the backward signal shrinks through each sigmoid layer, so the early layers barely move until the later layers have organized. With width 128 the same depth gets further in 5,000 passes (89.1% test). Tanh and ReLU leave the initial loss within 200 iterations.

3.10 MNIST: Network and Training

`mnist_dcn.py` fills in the starter and keeps its structure. `Net` is conv(5×5, 1→32, padding 2)–ReLU–maxpool(2)–conv(5×5, 32→64, padding 2)–ReLU–maxpool(2)–fc(3136→1024)–ReLU–dropout(0.5)–fc(1024→10)–softmax. Padding 2 keeps 28×28 and 14×14 through the convolutions, as with SAME padding in the original tutorial. Training uses mini-batches of 64, the starter's loss `NLLLoss(log(softmax + 1e-13))`, and 10 epochs. The command-line flags `--act`, `--init`, `--opt` and `--lr` select the variants of Section 3.12. Two changes from the starter: the normalization mean is 0.1307, the MNIST pixel mean (the starter has 0.01307), and `torch.manual_seed` is always called, because the starter seeds only CUDA while the weights are initialized on the CPU. The train loss is logged every 10 batches.

The starter's Adam learning rate of 0.01 does not train. After the first epoch every ReLU in conv2 and fc1 outputs zero on the monitoring batch, the network predicts a constant class, and test accuracy stays at chance level: 11.35% (the share of the digit 1) in every epoch except epoch 8, where the prediction switches to the digit 8 (9.74%). The baseline therefore uses Adam with lr = 10^{-3}. It reaches 98.80% test accuracy after one epoch and **99.17%** after 10 epochs; the best epoch is 99.32% (epoch 7). Figure 7 shows the training loss: it falls from 2.30 to about 0.1 within the first few hundred iterations and keeps decreasing slowly after that.

3.11 MNIST: Monitoring Weights, Activations and Test Error

After every epoch, and once before training, the code logs min, max, mean, standard deviation and a histogram for:

- the weights and biases of all four layers;

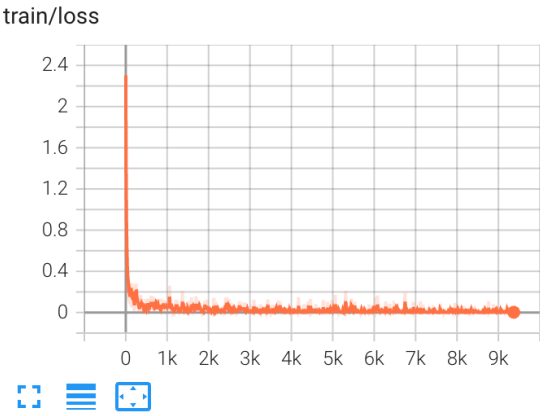


Figure 7: TensorBoard: training loss of the baseline (smoothing 0.6). The horizontal axis is iterations; 938 iterations make one epoch.

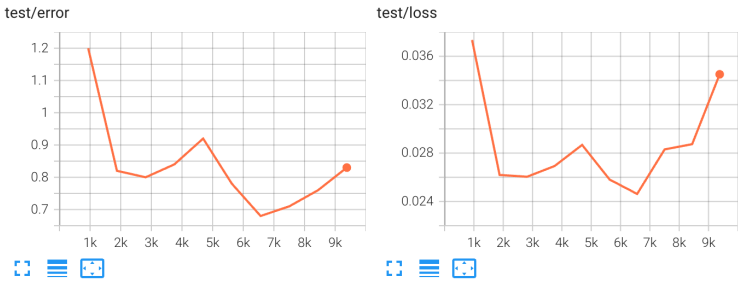


Figure 8: Test error (%) and test loss per epoch.

- the net inputs of conv1, conv2, fc1 and fc2;
- the ReLU outputs of conv1, conv2 and fc1;
- the max-pool outputs of both conv blocks.

Net inputs and activations are computed on a fixed batch of 1,000 test images in eval mode. Test loss, accuracy and error are logged every epoch. Figures 8–12 show the baseline. Figure 11 gives all four statistics for two layers, and Figure 12 gives the standard deviation of every monitored tensor.

Test error drops from 1.20% after epoch 1 to 0.82% after epoch 2 and then fluctuates between 0.68% and 0.92%. Test loss is lowest at epoch 7 (0.025) and rises to 0.035 by epoch 10, a mild sign of overfitting. The weights spread out during training: the standard deviation of fc1 grows from 0.010 at initialization to 0.051 and is still rising at epoch 10, though more slowly. The fc1 biases drift negative (mean -0.032), which turns more fc1 units off. The convolutional activations become sparser over training. The mean conv2 net input moves from -0.04 to -2.14, and the mean ReLU output of conv2 falls from 0.078 to 0.013, so most conv2 units are zero for any given input and a few respond strongly (max ≈ 7). The histograms show the same thing: a spike at zero with a long right tail. Max-pooling keeps the largest of four values, so its outputs have two to three times the mean of the ReLU outputs and a thinner spike at zero.

3.12 MNIST: Activations, Initialization and Optimizers

Table 3 lists all runs. The activation runs change only the nonlinearity (Adam, 10^{-3}, default PyTorch initialization). The initialization runs use plain SGD (lr 0.01), where the starting scale matters more than under

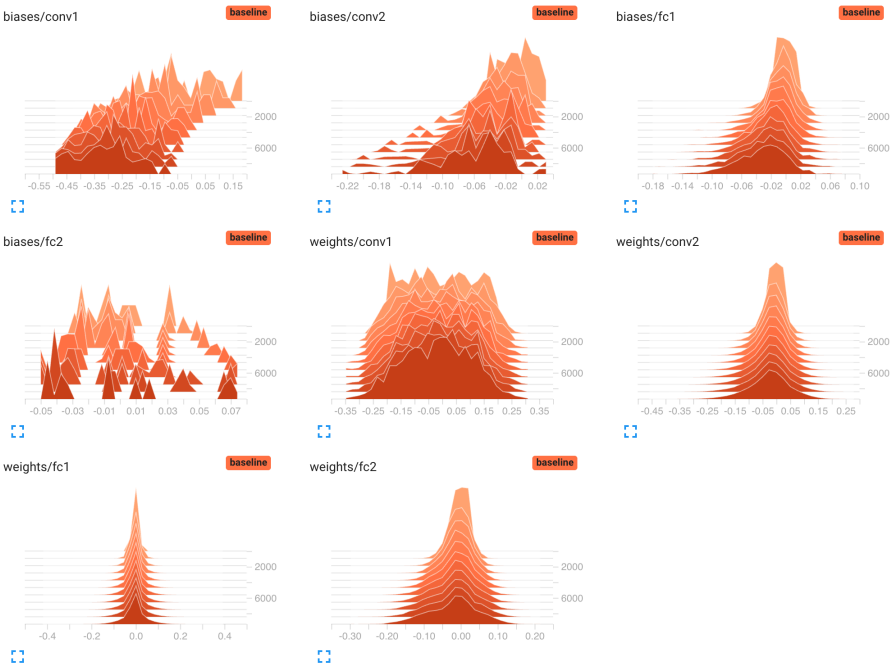


Figure 9: Histograms of biases and weights; front slices are later epochs.

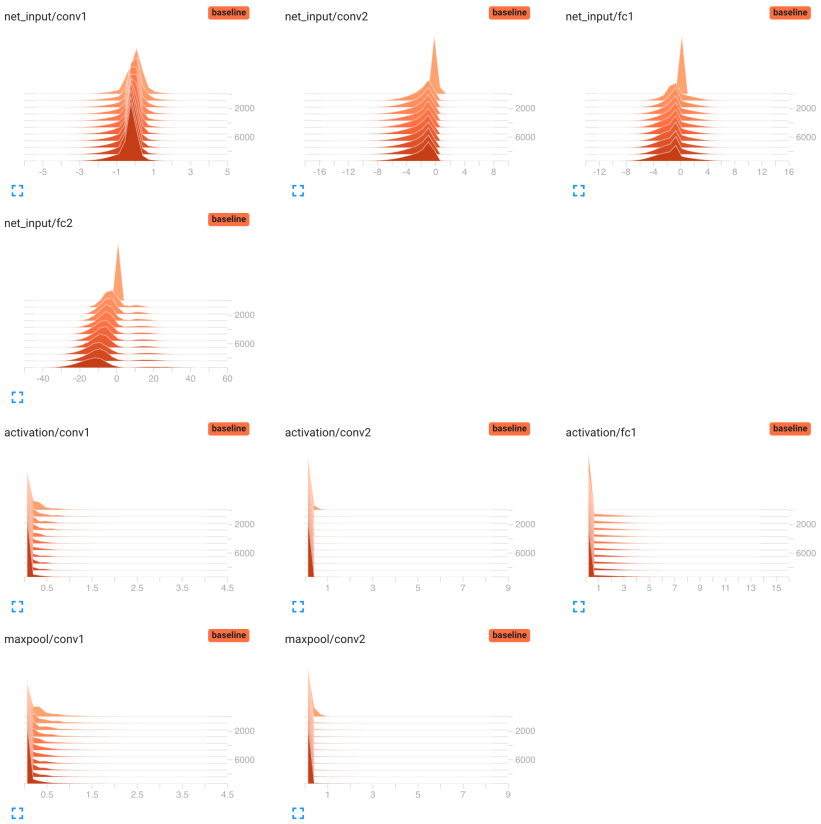


Figure 10: Histograms of net inputs (top), ReLU outputs and max-pool outputs (bottom).



Figure 11: Min/max/mean/std of fc1 weights and biases (top) and of the conv2 net input, ReLU output and max-pool output (bottom).

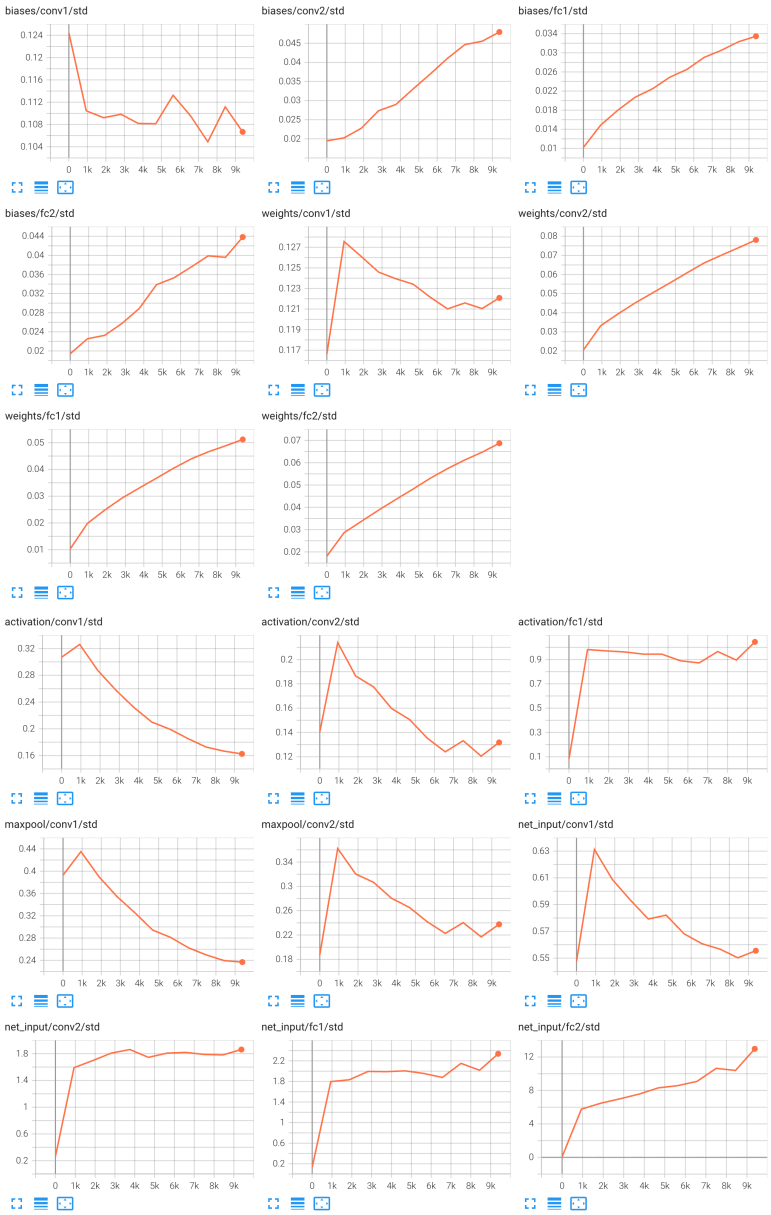


Figure 12: Standard deviation per epoch of every weight and bias (top) and of every net input, ReLU output and max-pool output (bottom).

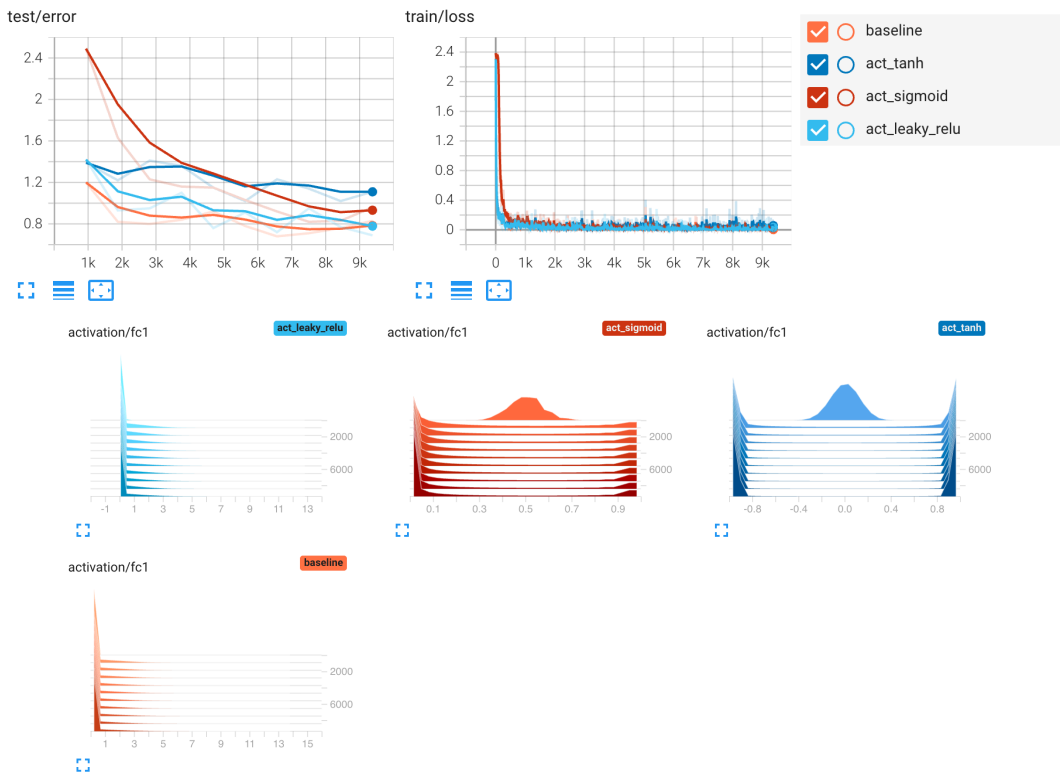


Figure 13: Activation functions: test error and training loss (top); histograms of fc1 activations (bottom). Bold curves use TensorBoard smoothing 0.6; faint curves are the raw values.

Adam. Xavier is uniform Glorot; Kaiming is He-normal for ReLU; biases are set to zero in both. The optimizer runs use ReLU and default initialization.

Table 3: MNIST test accuracy (%), 10 epochs.

Run	Activation	Init	Optimizer (lr)	Epoch 1	Epoch 10
baseline	ReLU	default	Adam (10^{-3})	98.80	99.17
act_tanh	tanh	default	Adam (10^{-3})	98.61	98.89
act_sigmoid	sigmoid	default	Adam (10^{-3})	97.51	99.04
act_leaky_relu	leaky ReLU (0.01)	default	Adam (10^{-3})	98.58	**99.31**
sgd	ReLU	default	SGD (0.01)	96.17	99.01
sgd_xavier	ReLU	Xavier	SGD (0.01)	96.69	98.97
sgd_kaiming	ReLU	Kaiming	SGD (0.01)	97.67	99.14
momentum	ReLU	default	SGD+momentum 0.9 (0.01)	98.50	99.14
adagrad	ReLU	default	Adagrad (0.01)	98.44	99.20
adam_lr1e-2	ReLU	default	Adam (0.01)	11.35	11.35

Activations. All four functions reach 98.9–99.3%. Leaky ReLU is best and tanh is worst. Sigmoid starts slowest (97.51% after epoch 1) and is within 0.25% of ReLU from epoch 7 on. The fc1 histograms in Figure 13 show why the squashing functions behave differently. At initialization, the sigmoid and tanh activations sit in a narrow band around 0.5 and 0 (std 0.07 and 0.12). By the end of training most units are

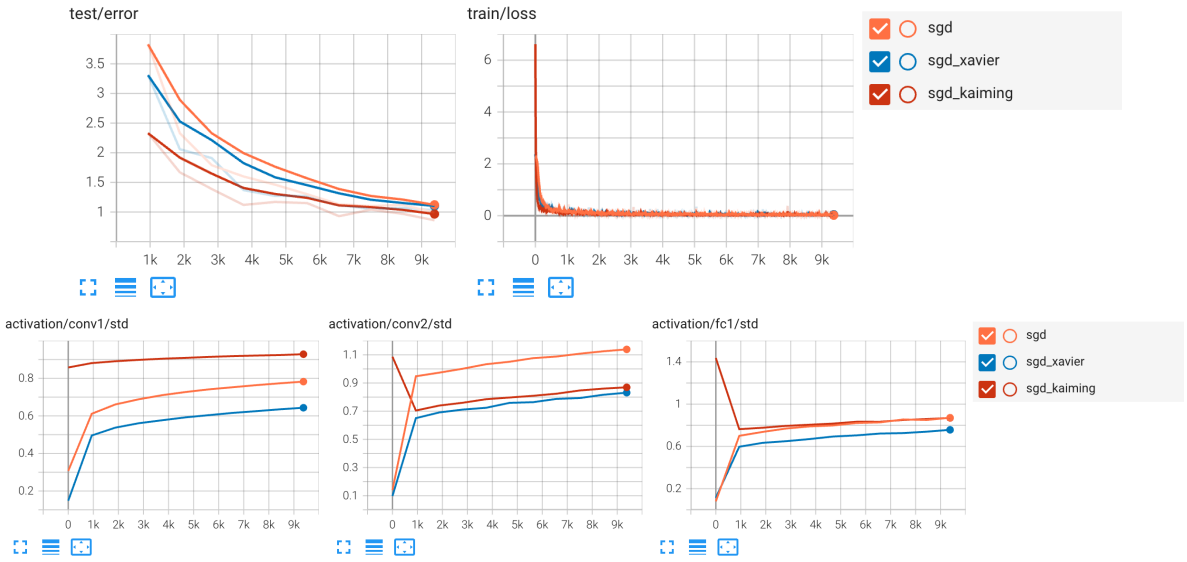


Figure 14: Initialization under SGD: test error and training loss, smoothing 0.6 with raw values faint (top); std of the ReLU outputs per layer (bottom).

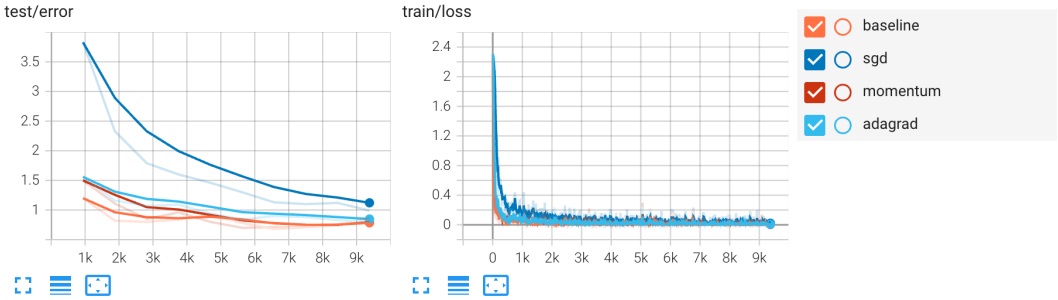


Figure 15: Optimizers: test error and training loss. Smoothing 0.6, raw values faint. Adam with lr 0.01 is left out because its 88.65% test error flattens the scale.

saturated, with mass piled at 0 and 1 for sigmoid and at ±1 for tanh (std 0.36 and 0.98). Saturated units pass almost no gradient. Adam's per-parameter scaling compensates enough for the network to train, but tanh's test error stays at 1.0–1.4% from epoch 2 on. ReLU and leaky ReLU do not saturate on the positive side and produce sparse activations; their histograms are nearly identical, since a slope of 0.01 for negative inputs barely changes the forward pass.

Initialization. Under plain SGD the initial activation scale determines how quickly the network starts learning (Figure 14). PyTorch's default (Kaiming-uniform with $a = \sqrt{5}$, i.e. weights in $\pm 1/\sqrt{\text{fan-in}}$) and Xavier both start with small activations: the per-layer std of the ReLU outputs at initialization is 0.08–0.31 for default and 0.10–0.15 for Xavier. He initialization preserves the variance through ReLU layers, and its activations stay at order 1 (0.86, 1.09, 1.44 for conv1, conv2, fc1). It reaches 97.67% after one epoch against 96.17% (default) and 96.69% (Xavier), and it finishes highest, at 99.14%. The cost is an initial training loss of 6.6 instead of 2.3: the ReLU gain is also applied to the linear output layer fc2, so the initial logits are large and the softmax starts confident and wrong. SGD recovers from this within a few dozen iterations.

Optimizers. With a working learning rate, all four optimizers end within 0.2% of each other (99.01–99.20%). They differ mainly in speed (Figure 15): Adam leads after one epoch (98.80%), momentum and Adagrad are close behind, and plain SGD needs 7 epochs to reach the error that Adam has after one. In the

Adam run with lr 0.01, Adam's update size is roughly lr per parameter regardless of the gradient's magnitude, so in the first steps every fc1 weight moves by about 0.01, against an initial std of 0.010. The fc1 weight std jumps to 0.054 within the first epoch, the pre-activations of conv2 and fc1 are pushed negative everywhere, and all of those ReLUs die (max activation exactly 0 from epoch 1 on). A dead ReLU has zero gradient, so the network cannot recover, and only the fc2 bias keeps learning, which yields a constant prediction.

3.13 Discussion

Depth with sigmoid stalls in both parts. A 10-layer sigmoid network on Moons and a 6-layer sigmoid network on Digits barely train, because $\sigma' \leq 0.25$ shrinks the gradient at every layer; tanh and ReLU networks of the same depth train normally. On MNIST, sigmoid and tanh units end up saturated, and ReLU units die when the Adam step is too large (lr 0.01). Under plain SGD, He initialization gives the fastest start. With a working learning rate, SGD, momentum, Adagrad and Adam all finish at 99.0–99.2%, and all working MNIST runs at 98.9–99.3%.

On the small datasets, extra capacity fits noise. Make-Moons needs 3–4 tanh units, and wider or deeper networks add islands and protrusions around single points, lowering test accuracy from 0.967 to about 0.95. On Digits, one hidden layer already reaches 97–98%. ReLU boundaries are piecewise linear (Figures 2 and 4). In the DCN, conv2 net inputs drift negative during training, so most conv2 units are off for a given image and a few respond strongly.

Code and TensorBoard logs are in the attached `sj186-assignment1.zip`.